# Graph Representations

We primarily represent graphs using two ways:

1. Adjacency matrix
2. Adjacency list

Let's explore what they are.

## Adjacency Matrix

An adjacency matrix is a common way to represent a graph as a matrix. It is a square matrix where the rows and columns represent the vertices of the graph, and the entries (elements) of the matrix indicate whether there is an edge between the corresponding vertices.

In an undirected graph, the edges have no direction, meaning they can be traversed in both directions between two vertices. On the other hand, in a directed graph, the edges have a direction, indicating a one-way relationship between two vertices.

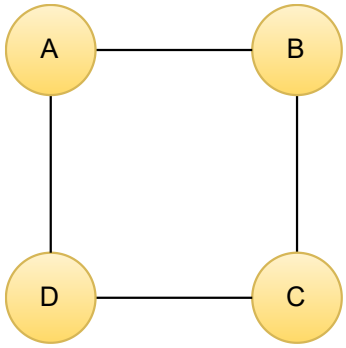
As there are two major types of graphs directed graph and undirected graph. Let's see how the adjacency matrix works for both types of graphs:

### Adjacency matrix for undirected graphs

In an undirected graph with N vertices, the adjacency matrix A will be an N x N matrix. For an undirected edge between vertices i and j, the corresponding entries in the matrix (A[i][j] and A[j][i]) will have a value of 1, indicating the presence of an

edge. If there is no edge between vertices i and j, the matrix entries will have the value of 0.

Example of an undirected graph with 4 vertices (A, B, C, D) and 4 edges (A-B, B-C, C-D, D-A):




Undirected Graph with 4 Edges (A-B, B-C, C-D, D-A)

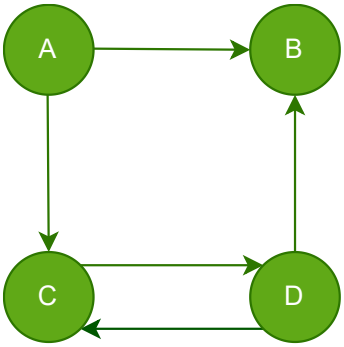
| | A | B | C | D |
|---|---|---|---|---|
| A | 0 | 1 | 0 | 1 |
| B | 1 | 0 | 1 | 0 |
| C | 0 | 1 | 0 | 1 |
| D | 1 | 0 | 1 | 0 |

Adjacency Matrix for the Above Graph with 4 Edges (A-B, B-C, C-D, D-A)

## Adjacency matrix for directed graphs

In a directed graph with N vertices, the adjacency matrix A will also be an N x N matrix. For a directed edge from vertex i to vertex j, the corresponding entry in the matrix ($A[i][j]$) will have the value of 1, indicating the presence of an edge from i to j. If there is no edge from vertex i to vertex j, the matrix entry will have the value of 0.

Example of a directed graph with 4 vertices (A, B, C, D) and 5 directed edges (A->B, A->C, C->D, D->B, D->C):




Directed Graph with 5 Edges (A->B, A->C, C->D, D->B, D->C)

| | A | B | C | D |
|---|---|---|---|---|
| A | 0 | 1 | 1 | 0 |
| B | 0 | 0 | 0 | 0 |
| C | 0 | 0 | 0 | 1 |
| D | 0 | 1 | 1 | 0 |

The above figure explains the adjacency matrix of the directed graph in such a way that there is an edge between vertices A-C and A-B so 1 is placed there.

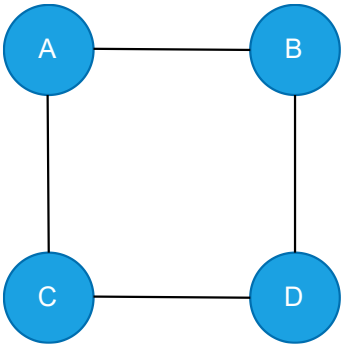
## Adjacency List

In linked list representation, an adjacency list is used to store the graph. An adjacency list is a common way to represent the connections between vertices in a graph. It is used in both directed and undirected graphs, but the way edges are stored and described differs slightly between the two types. In an adjacency list, each vertex is associated with a list of its neighboring vertices directly connected to it.

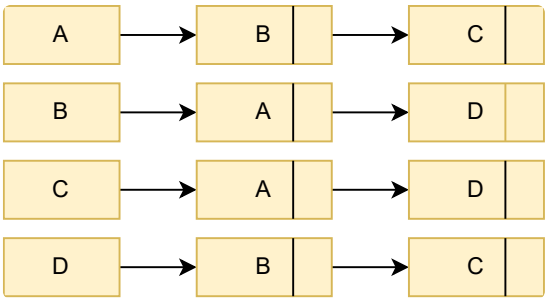
### Representing undirected graph using adjacency list

In an undirected graph, the edges between vertices have no direction. If vertex A is connected to vertex B, then vertex B is also connected to vertex A. As a result, the

adjacency list for an undirected graph is symmetric. Here is an example of a undirected graph with four vertices (A, B, C, D) and four edges.



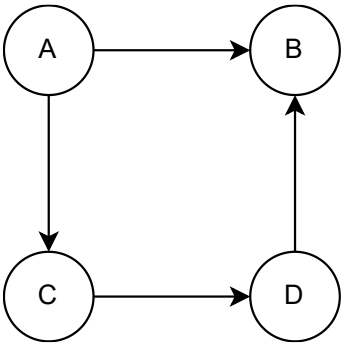

Here is the adjacency list for the above-undirected graph. From vertex A there is an edge to vertex B and C in the graph. So in the adjacency list, there are two nodes from node A.



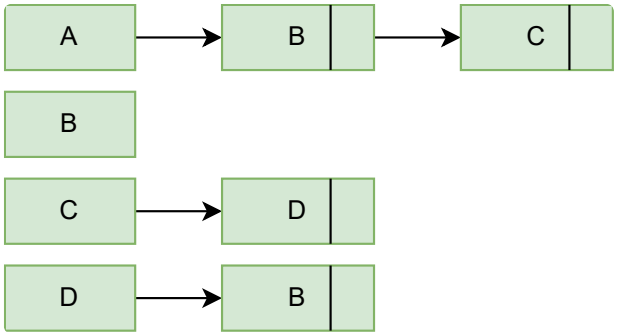

## Representing directed graphs using adjacency list

In a directed graph, the edges between vertices have a direction. If vertex X is connected to vertex Y, it does not necessarily mean that vertex Y is connected to vertex X. As a result, the adjacency list for a directed graph is not symmetric.

Example of a directed graph with 4 vertices (A, B, C, D) and 4 directed edges (A->B, A->C, C->D, D->B):



Here is the adjacency list for the above directed graph. From vertex A there is an edge to vertex B and C in the graph. So in the adjacency list, there are two nodes from node A. From vertex B there is no edge coming out so the adjacency list contains no further node from node B.




← Previous

Introduction To Graph

Next →

Graph As An Abstract Data Type (ADT)

Mark as Completed